University of East Anglia Norwich
School of Computing Sciences

Advanced Web Development

Excise 2.0: REACT Basics – Component, Props and JSX

In this lab, we will be using React 19, the latest version of the library. This session assumes you are already familiar with TypeScript and using the command-line console. Before starting, please ensure you have completed the TypeScript lab exercises.

If you need a refresher, you can find official documentation on the TypeScript Website https://www.typescriptlang.org and a quick overview of how it relates to JavaScript in the TypeScript in 5 Minutes guide https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html. For a quick guide on console commands and usage, check out this CLI Command Cheat Sheet https://gist.github.com/bradtraversy/cc180de0edee05075a6139e42d5f28ce.

Historically, React components (https://react.dev/reference/react/Component) were built using JavaScript classes. However, modern React components are constructed using JavaScript functions. Anyone working with older versions of React or legacy class components is strongly encouraged to migrate to this functional approach. You can find more details and migration strategies in the React Component Alternatives guide https://react.dev/reference/react/Component#alternatives.

In React, a component is essentially a JavaScript function. By convention, component names must always begin with a capital letter. A component must return a value to be rendered on the page. While you can think of this as standard HTML and CSS, React utilises JSX (JavaScript XML). Since we are using TypeScript, our files will use the `.tsx` extension instead of `.jsx`.

To be valid, the JSX code returned by a component must be wrapped in a single root element. This can be any semantic HTML tag, such as `<div>`, `<section>`, `<nav>`, or `<aside>`. While it is easy to just default to a `<div>` as the root element, doing this everywhere can lead to a messy, deeply nested DOM structure. To avoid redundant markup, you should choose your root elements carefully. To learn more about how this works, check out the official guide on writing markup with JSX: https://react.dev/learn/writing-markup-with-jsx

Our first exercise is to build a React application that displays a list of Stephen King's books. Normally, you would fetch this data by making an HTTP request to an API endpoint. However, for this introductory exercise, I have already prepared the data for you. You can download the dataset directly from Blackboard.

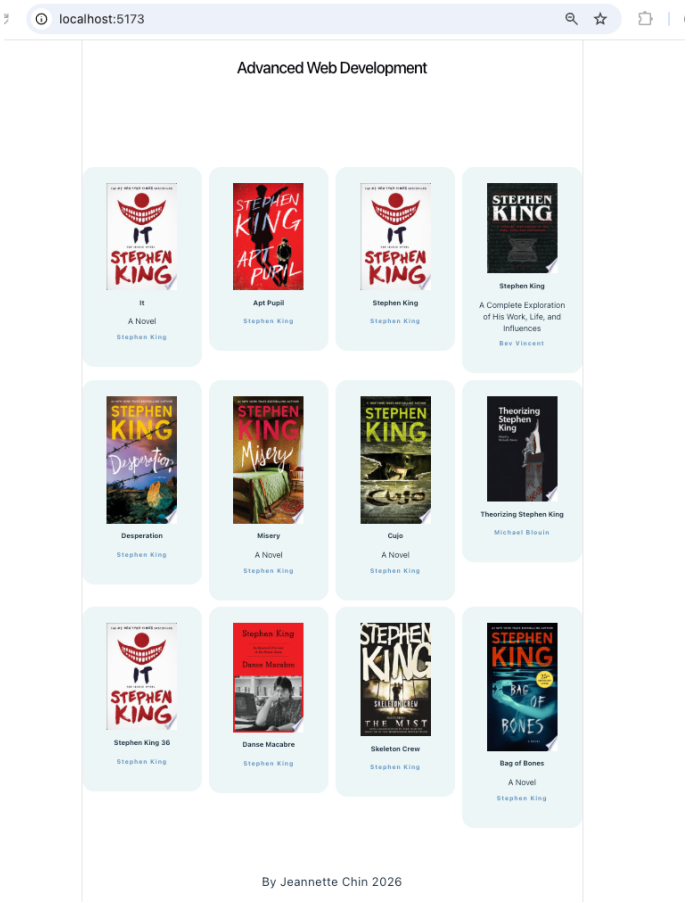
Let's get started!

Learning Objectives:

- Able to code REACT components using JavaScript function.
- Able to code JSX for rendering components using static data.
- Able to handle JSON data.
- Able to use spread operators to unpack JSON data
- Able to code JSX for rendering components using JSON
- Practice writing CSS rules to style the webpage.

The Aim of this lab exercise is to create an output / webpage of REACT Application using TypeScript and looks similar to below. JSON format of the book data will be rendered on a responsive webpage. Each book will have an image, title, and authors name.

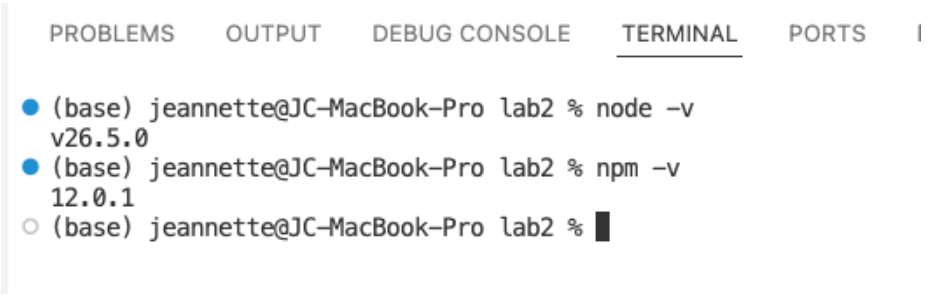
Final output:



Pre-requisite:

In order to follow the guide, I will recommend you use the same environment as mine on your machine. This is to avoid any incompatibility issues along the way.

1. Check `node` and `npm` versions. My `node` version is – v26.5.0 , and `npm` is 12.0.1



2. Install the latest package - create-vite, we will use this to create the app.

```
[CMP-25JCMBP:~] cew19wcu% npm i create-vite@latest
```

Result:

```
[[CMP-25JCMBP:~] cew19wcu% npm i create-vite@latest

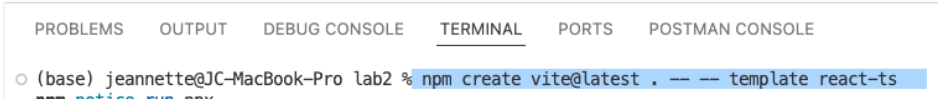
added 3 packages, removed 1 package, and audited 120 packages in 6s

25 packages are looking for funding
  run `npm fund` for details
```

Tasks:

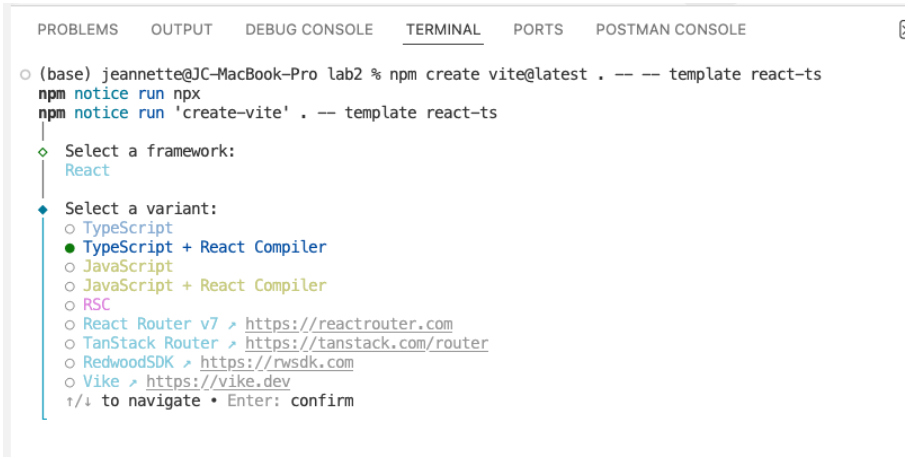
1. Create a folder called “lab2” and open this folder on VSCode
2. On the terminal window type

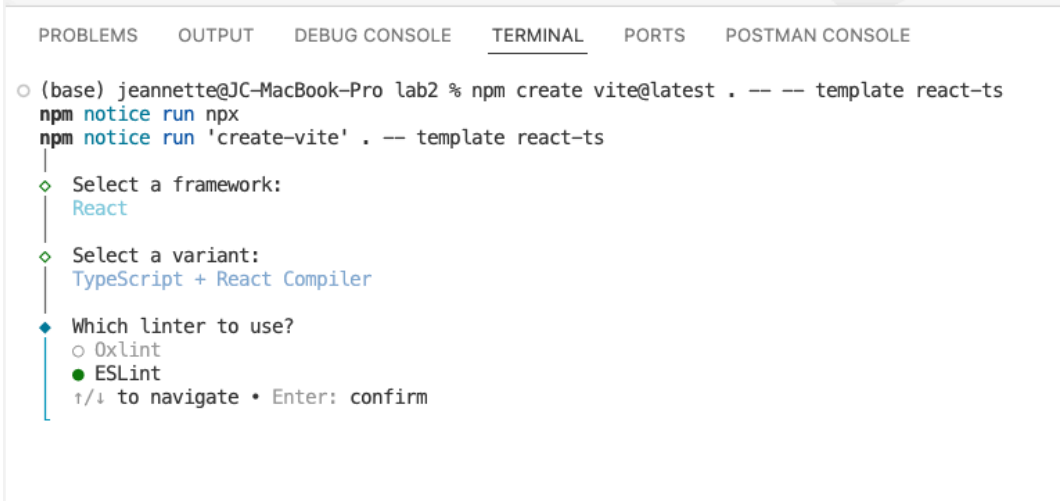
```
npm create vite@latest . -- -- template react-ts
```

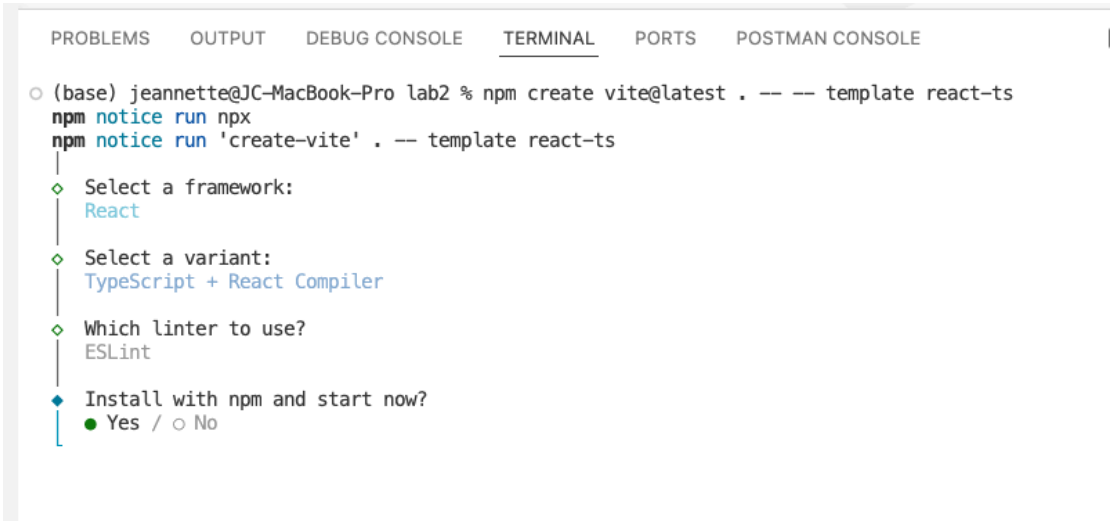


3. Follow the instructions on the screen:

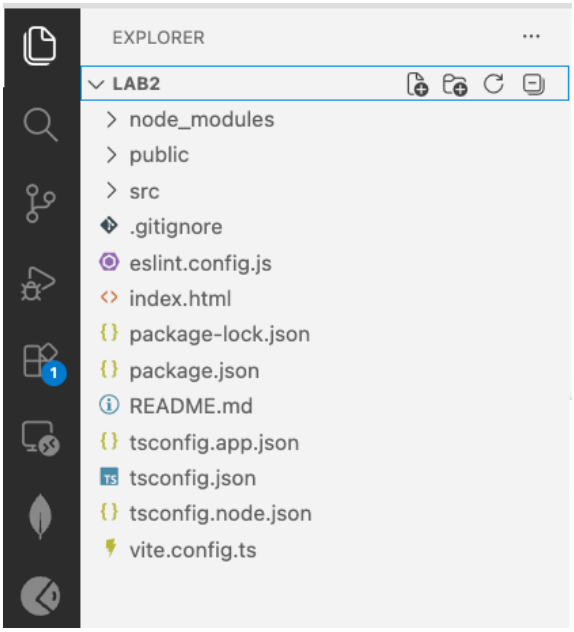



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
(base) jeannette@JC-MacBook-Pro lab2 % npm create vite@latest . -- -- template react-ts
npm notice run npx
npm notice run 'create-vite' . -- template react-ts
Select a framework:
React
Select a variant:
TypeScript
TypeScript + React Compiler
JavaScript
JavaScript + React Compiler
RSC
React Router v7 https://reactrouter.com
TanStack Router https://tanstack.com/router
RedwoodSDK https://rwsdk.com
Vike https://vike.dev
↑/↓ to navigate • Enter: confirm

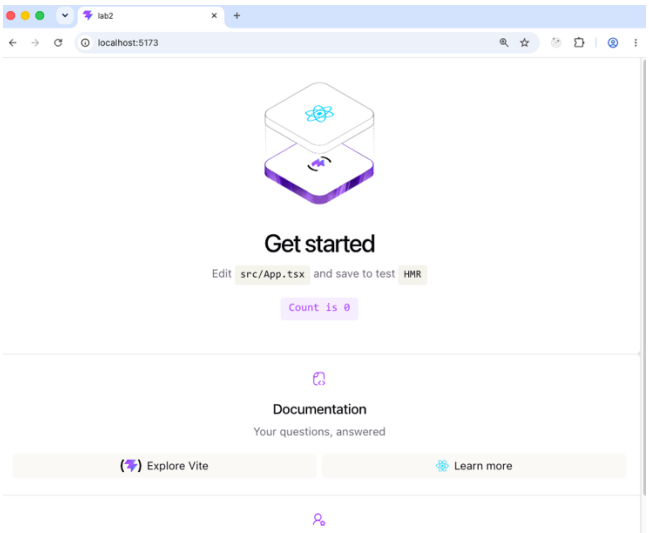

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
(base) jeannette@JC-MacBook-Pro lab2 % npm create vite@latest . -- -- template react-ts
npm notice run npx
npm notice run 'create-vite' . -- template react-ts
Select a framework:
React
Select a variant:
TypeScript + React Compiler
Which linter to use?
Oxlint
ESLint
↑/↓ to navigate • Enter: confirm


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
(base) jeannette@JC-MacBook-Pro lab2 % npm create vite@latest . -- -- template react-ts
npm notice run npx
npm notice run 'create-vite' . -- template react-ts
Select a framework:
React
Select a variant:
TypeScript + React Compiler
Which linter to use?
ESLint
Install with npm and start now?
Yes / No

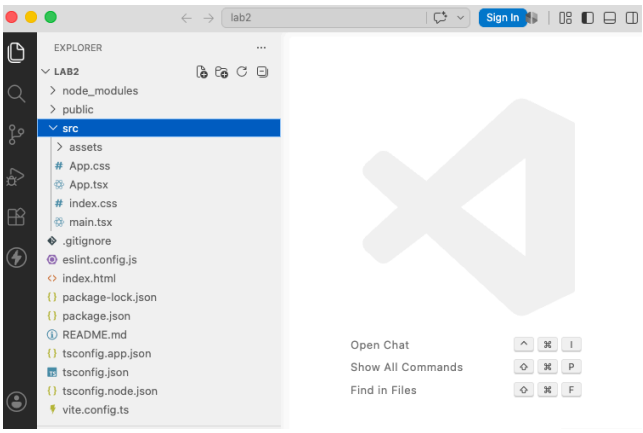
4. Examine the folder structure on VSCode:



5. Check it out on Chrome: http://localhost:5173/



6. We will place our code inside the 'src' folder. There is an 'assets' folder inside this 'src' folder where we can put our images or assets in here. Note that the index.html page is in the root folder. Vite also provides tools to help deployment tasks. Once we have finished the coding we can build the entire project for deployment. One thing to note is that the file extension is .tsx as we are using TypeScipt. More information can be found here: https://react.dev/learn/typescript



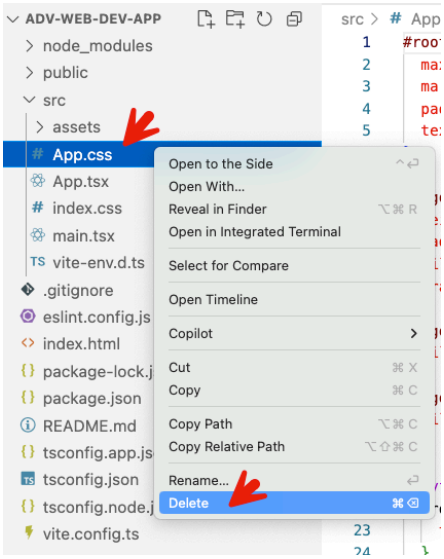
7. Inspect the 'package.json' file and you will see the dependencies and their version for this project. My version is shown below for your information. By the time you create your project, your versions should be higher than mine.

package.json > ...

```
{
  "name": "lab2",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.7",
    "react-dom": "^19.2.7"
  },
  "devDependencies": {
    "@babel/core": "^7.29.7",
    "@eslint/js": "^10.0.1",
    "@rolldown/plugin-babel": "^0.2.3",
    "@types/babel__core": "^7.20.5",
    "@types/node": "^24.13.2",
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.3",
    "babel-plugin-react-compiler": "^1.0.0",
    "eslint": "^10.6.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.3",
    "globals": "^17.7.0",
    "typescript": "~6.0.2",
    "typescript-eslint": "^8.62.0",
    "vite": "^8.1.1"
  }
}

```

8. As you can see, Vite generates a scaffold REACT app for us so that we can just get on with coding our application. However, there are files we don't need and files we may want to modify. So, for now, we will remove the `App.css` file completely, as we don't need it.



9. Open the `App.tsx` file on VS Code. Notice that the component here is called 'App' (recall in React, a component is basically a JS function). In REACT, component is written in Title case, so we will follow this convention. We export the component using Export default keywords. JSX code is from line 7 to line 40 below:

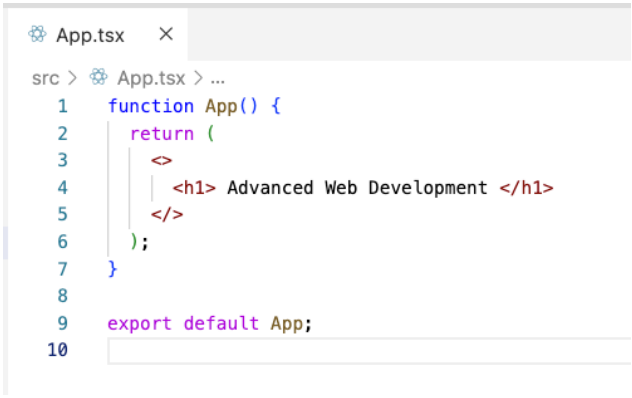
```
7   function App() {
8     const [count, setCount] = useState(0)
9
10    return (
11      <>
12        <section id="center">
13          <div className="hero">
14            <img src={heroImg} className="base" width="170" height="179" alt="" />
15            <img src={reactLogo} className="framework" alt="React logo" />
16            <img src={viteLogo} className="vite" alt="Vite logo" />
17          </div>
18          <div>
19            <h1>Get started</h1>
20            <p>
21              Edit <code>src/App.tsx</code> and save to test <code>HMR</code>
22            </p>
23          </div>
24          <button
25            type="button"
26            className="counter"
27            onClick={() => setCount((count) => count + 1)}
28          >
29            Count is {count}
30          </button>
31        </section>
32
33        <div className="ticks"></div>
34
35
36        <div className="ticks"></div>
37        <section id="spacer"></section>
38      </>
39    )
40  }
41
42  export default App
43
```

10. Remove everything as shown below, and write some words wrap inside a pair of <h1> tags as follow:

    Before:



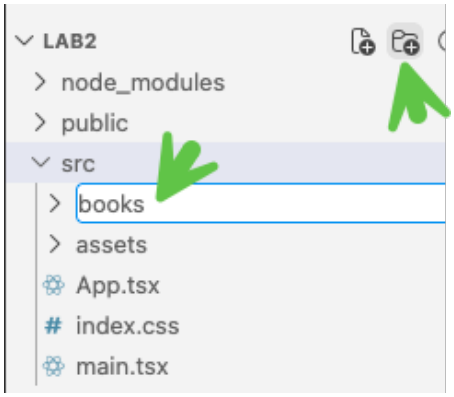
    After:



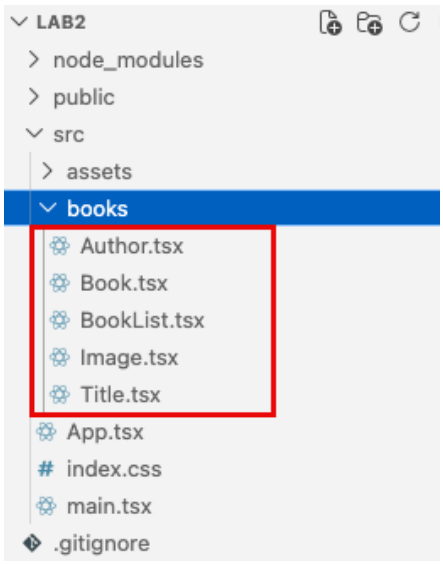
11. Check the result on Chrome, you should see this:



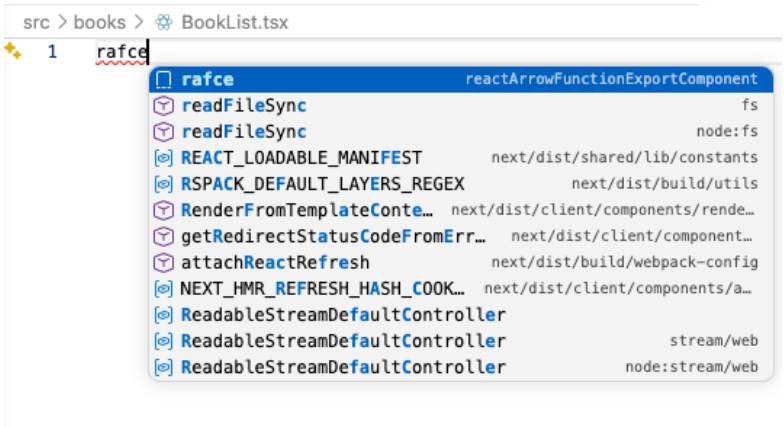
12. Inside the '`src`' folder, create a new subfolder called '`books`'



13. Inside this new subfolder '`books`', create five files – `BookList, Book, Title, Author`, and `Image`. Notice that the file extension is `'.tsx'`



14. Open the `BookList.tsx`, and define this Component using JavaScript function. If you have installed snippets plugin, simply type:

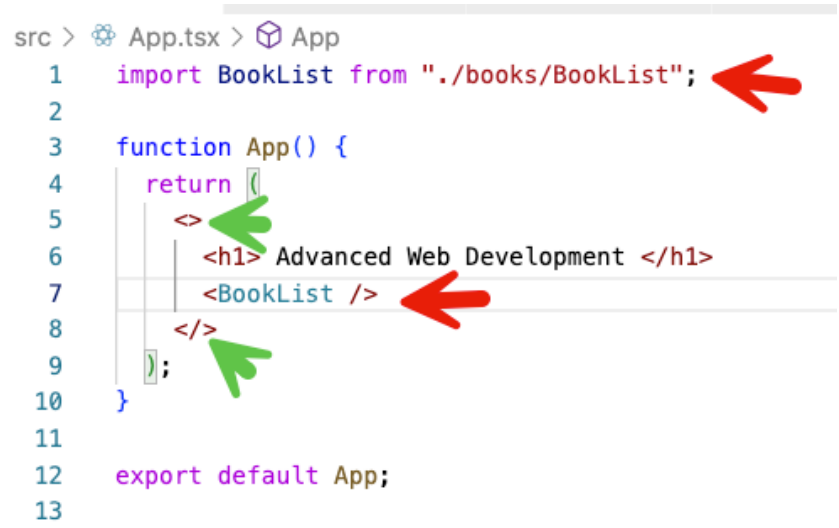


15. Below is my component. Notice I use `<section>` as the root (i.e. parent) element here (line 5), JSX markup code is from line 5 to line 7.

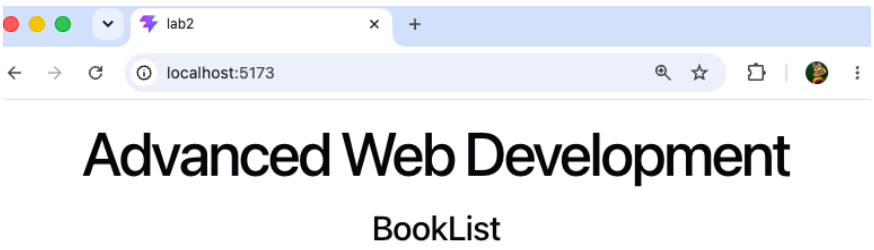
src > books > BookList.tsx > ...

```
1   import React from "react";
2
3   const BookList = () => {
4     return (
5       <section>
6         <h2>BookList</h2>
7       </section>
8     );
9   };
10
11  export default BookList;
12
```

16. Now to add this Component to `App.tsx` (line 7), we simple refer to its name in our JSX markup. Notice that I have imported the file in `App.tsx` (line 1) otherwise REACT will complain. In order to avoid using too many `<div>`s , REACT provides a handy way to declare an empty root element, here is an example how to use them (line 5 and line 8).



17. You should see the output immediately display on the webpage once you have saved your file:



18. Next, repeat Task (14) for 'Book', 'Title', 'Image' and 'Author' Components. The root element for Book Component should be `<article>`, Title is `<h3>`, Author is `<h4>` and Image is `<div>`. You can use any text you wish as the placeholder.

19. Once you finish, add the 'Image', 'Title' and 'Author' Components to Book Component (see line 8 to line 10). Don't forget to import the files (see line 1 to line 3), as shown below:

src > books > Book.tsx > ...

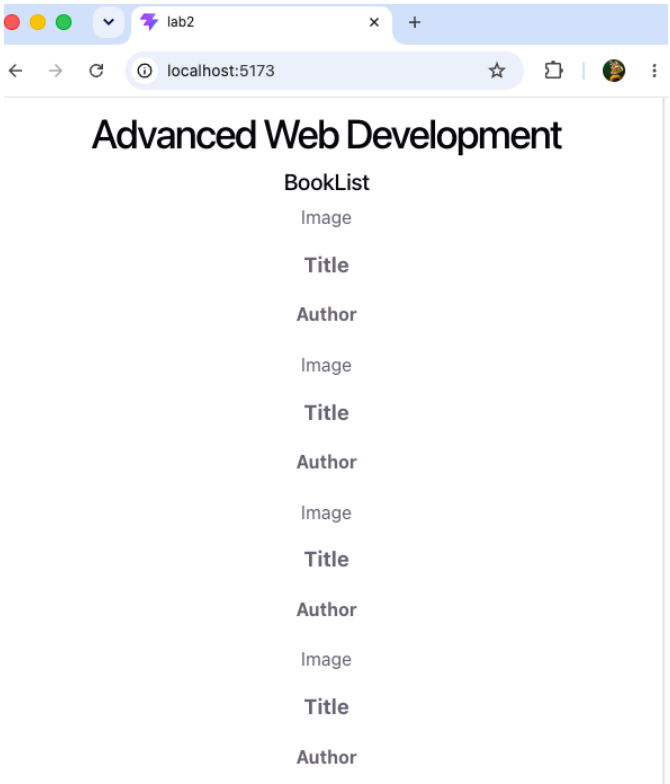
```
1  import Title from "./Title";
2  import Image from "./Image";
3  import Author from "./Author";
4
5  const Book = () => {
6    return (
7      <article>
8        <Image />
9        <Title />
10       <Author />
11     </article>
12   );
13 };
14
15 export default Book;
16
```

20. Next, we will want to add some 'Book' to the 'BookList', so let's just do that. I will add five books (line 7 to line 11)

src > books > BookList.tsx > ...

```
1   import Book from "./Book";
2
3   const BookList = () => {
4     return (
5       <section>
6         <h2>BookList</h2>
7         <Book />
8         <Book />
9         <Book />
10        <Book />
11        <Book />
12      </section>
13    );
14  };
15
16  export default BookList;
17
```

21. Save the file. What do you see on your browser? Here is how it looked on my browser:



22. Now that we have our component structure set up, we will move on to code our 'Image' Component. The role of this 'Image' Component is to render the image of the book. So, let's code this Image Component. For this task, I have hardcoded the URL – 'https://books.google.com/books/content?id=S85NCwAAQBAJ&printsec=frontcover&img=1&zoom=1&edge=curl&source=gbs_api' but you can use any image URL if you wish. Replace the root element - `<div>` tag with `<img>` and code the image element, as shown below:

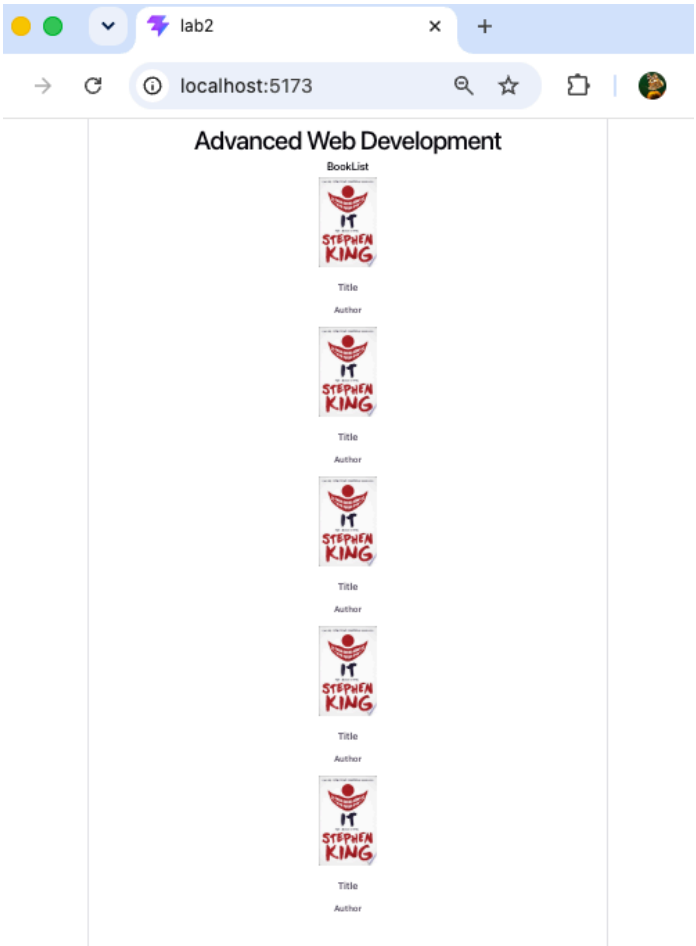
src > books > Image.tsx > ...

```tsx
const Image = () => {
  return (
    <img
      src="https://books.google.com/books/content?id=S85NCwAAQBAJ&printsec=frontcover&img=1&zoom=1&edge=curl&source=gbs_api"
      alt="Stephen King"
    />
  );
};

export default Image;

```

23. Save the file, and what do you see on your browser? Below is the output of my browser. As you can see, I have added 5 'Book' in my 'BookList' (see task 20 above), and because each 'Book' has the same URL, there are 5 identical images show on the page. Obviously if you are using a different image URL, then yours will look different!



24. Now let's code the Author Component. We could hardcode a name in this Component, however, wouldn't it be nice if we could, say, render a variable with some value, dynamically. The first thing we could try is to create a variable in this Component (line 1), and pass this variable to JSX (line 3), as shown below. Notice I use curly bracket for inserting the variable, this is how we pass variable to JSX. JSX treats the JavaScript code written inside a pair of curly brackets as expressions.

src > books > Author.tsx > ...

```
1   const author = "Jeannette Chin";
2   const Author = () => {
3     return <h4>{author}</h4>;
4   };
5
6   export default Author;
7
```

Question: what is the data type of this 'author'?

25. When you save the file, you will see my name (or whatever value of you have assigned to your variable) being rendered as the author on your webpage.

26. One of the core concepts in React is properties, universally referred to as 'props'. Props are passed down from a parent component to a child component. In React, this follows a one-way data flow, meaning data can only flow downward from the top of the component tree (parent to child) and never in reverse. So, in our current app structure, our Book component is the parent of our Author, Title and Image components. Let's pass a value as props from 'Book' component (parent) to 'Author' component (child). Below figure shows the variable author (line 5) passes down as props (name) to the child component 'Author' (line 12):

src > books > Book.tsx > ...

```
1   import Title from "./Title";
2   import Image from "./Image";
3   import Author from "./Author";
4
5   const author = "Jeannette Chin";
6
7   const Book = () => {
8     return (
9       <article>
10        <Image />
11        <Title />
12        <Author name={author} />
13      </article>
14    );
15  };
16
17  export default Book;
18
```

Notice that the variable 'author' (line 5) is placed outside the function component (line 7), and I use 'name' as my props name (line 12), and the variable 'author' is passed inside a pair of curly brackets. Notice that typescript is complaining (line 12) because the child component "Author" does not have a property called 'name' yet. We will fix this later.

27. Now to retrieve this 'props' from the 'Author' (child) component, we simply pass it as props (line 2) as argument. Notice that the naming of the property must be exactly the same as defined in the parent (task 26). TypeScript requires us to declare the type of the props, in our case, it is merely a string (line 2).

```
src > books > Author.tsx > ...
1   // const author = "Jeannette Chin";
2   const Author = ({ name }: { name: string }) => {
3     return <h4>{name}</h4>;
4   };
5
6   export default Author;
7
```

28. Now save the files and observe the output on your webpage.

29. It's a good time to add some CSS to our webpage. We can use inline style even though you might encounter previously that this was discouraged for all the good reasons. I think given we have already imported the '`index.css`' in our '`main.tsx`', we can just modify this CSS file and add the style rules that we want. Your task is to create some CSS rules and style these books to your liking. Place your CSS rules in '`index.css`' or use TailwindCSS classes. Use '`className`' to refer to CSS class in JSX code, see line 7. Here I have created a CSS class called 'booklist' in my '`index.css`', and used '`className`' to refer to this CSS class in '`BookList.tsx`'. I have also tidied the markups as well, see figure below:

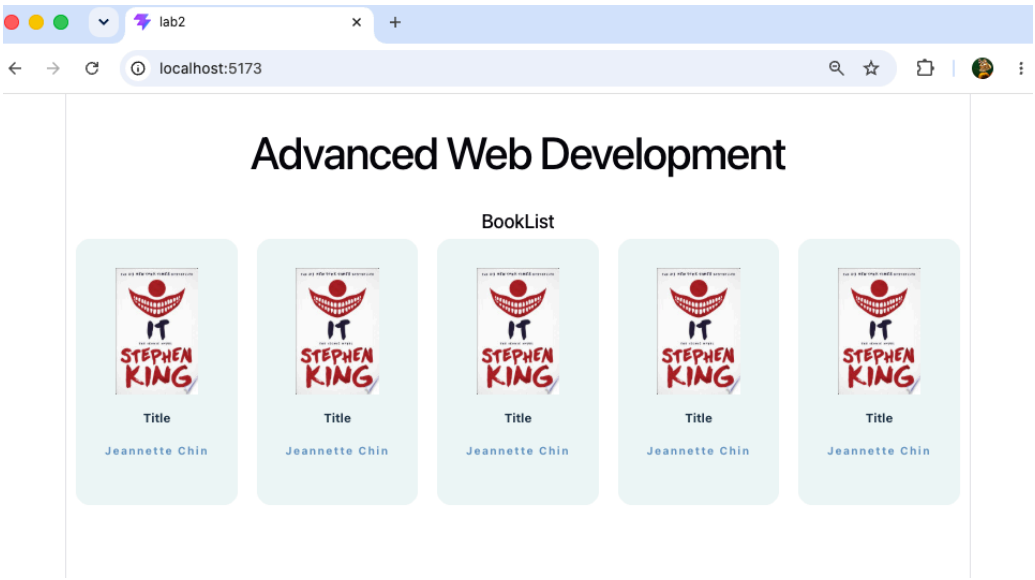
src > books > BookList.tsx > ...

```tsx
import Book from "./Book";

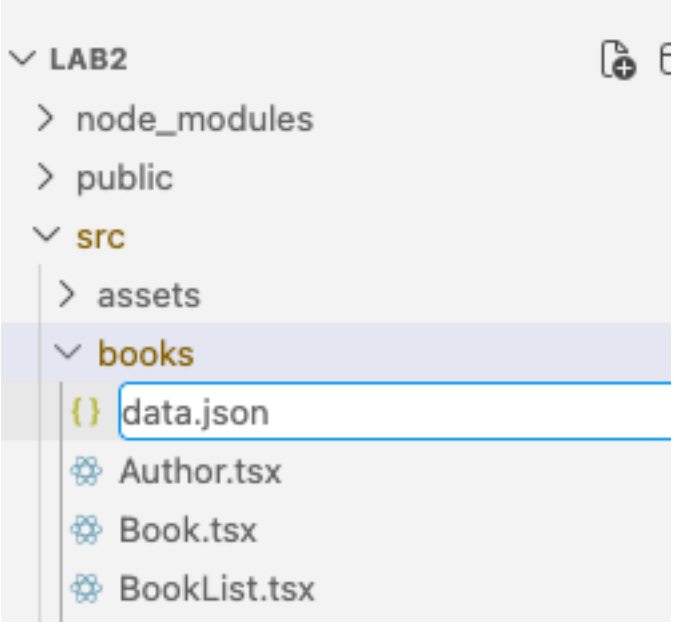
const BookList = () => {
  return (
    <section>
      <h2>BookList</h2>
      <div className="booklist">
        <Book />
        <Book />
        <Book />
        <Book />
        <Book />
      </div>
    </section>
  );
};

export default BookList;
```

30. Below is my UI output for your reference. Suggest using flex box or grid layout for your styling to achieve responsiveness, I will leave this to you.



31. Now that we have learned how pass props down to a child from a parent in React. It's time to look at how to retrieve data from a file and render it on the page. If you have not already done so, create a JSON file called '`data.json`' in 'src/books' folder:

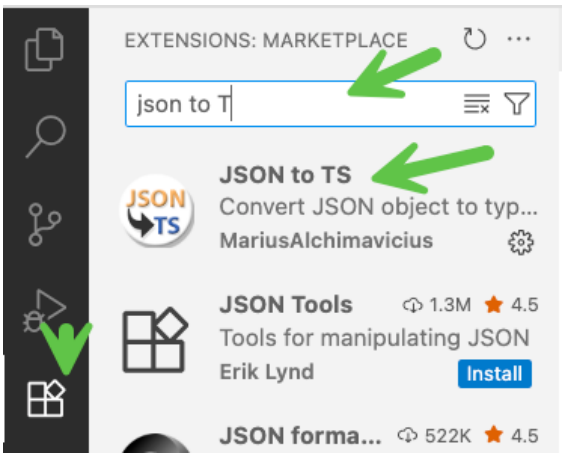


32. On Blackboard you will find a file named "`data.json`", you can either (1) copy the content in this file and paste it to the '`data.json`' that you just created, or (2) if you wish, replace the one you just created with the one on Blackboard. It is up to you how you want to do this.

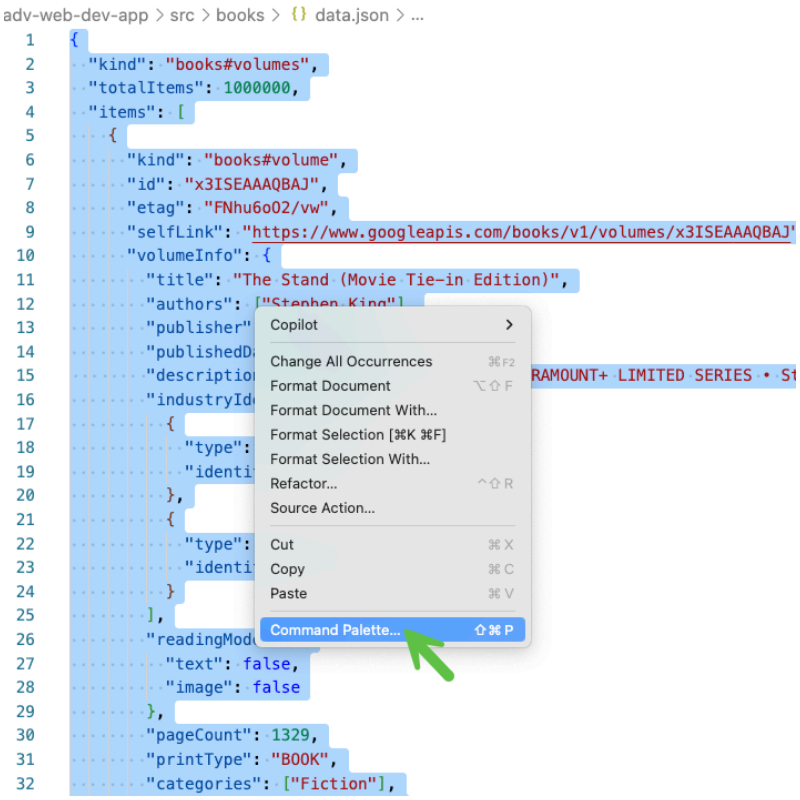
33. Now examine the content in '`data.json`', we know it's written in JSON format. You can fetch this data via Google API: https://www.googleapis.com/books/v1/volumes?q=stephen+king&key=YOUR-API-KEY. Google has changed the access which now requires an API key. You can apply for this API KEY on Google Cloud, more information can be found here: https://developers.google.com/books/docs/v1/using. For now, we will work on this static data I have prepared for you. As you will see, there is a lot of information here, but we are only interested in the books' details, which you can find inside the 'items' array (see line 4). We are interested in the book title, subtitle, authors, and the image (thumbnail) of this book. From the data, we can see that the book title, subtitle and authors information can be found inside the 'volumeInfo' object (see line 10), and the image (thumbnail) URL information is inside the "imageLinks" object (see line 47).

```
{
  "kind": "books#volumes",
  "totalItems": 1000000,
  "items": [
    {
      "kind": "books#volume",
      "id": "S85NCwAAQBAJ",
      "etag": "7RmPtAl9wv4",
      "selfLink": "https://www.googleapis.com/books/v1/volumes/S85NCwAAQBAJ",
      "volumeInfo": {
        "title": "It",
        "subtitle": "A Novel",
        "authors": [
          "Stephen King"
        ],
        "publisher": "Simon and Schuster",
        "publishedDate": "2016-01-05",
        "description": "Stephen King's classic #1 New York Times bestseller and the basis fo
        "industryIdentifiers": [
          {
            "type": "ISBN_13",
            "identifier": "9781501142970"
          },
          {
            "type": "ISBN_10",
            "identifier": "1501142976"
          }
        ],
        "readingModes": {
          "text": false,
          "image": false
        },
        "pageCount": 1168,
        "printType": "BOOK",
        "categories": [
          "Fiction"
        ],
        "averageRating": 4,
        "ratingsCount": 23,
        "maturityRating": "NOT_MATURE",
        "allowAnonLogging": false,
        "contentVersion": "0.8.3.0.preview.0",
        "panelizationSummary": {
          "containsEpubBubbles": false,
          "containsImageBubbles": false
        },
        "imageLinks": {
          "smallThumbnail": "http://books.google.com/books/content?id=S85NCwAAQBAJ&printsec=
          "thumbnail": "http://books.google.com/books/content?id=S85NCwAAQBAJ&printsec=front
        },
        "language": "en",
        "previewLink": "http://books.google.co.uk/books?id=S85NCwAAQBAJ&printsec=frontcover&
        "infoLink": "http://books.google.co.uk/books?id=S85NCwAAQBAJ&dq=stephen+king&hl=&sou
```

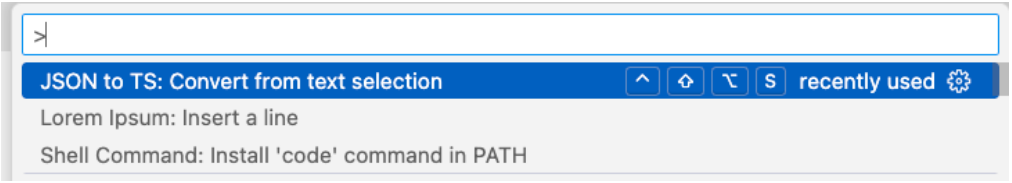

34. We will need to write the types for the JSON data so that we can use it in the app. The easier way to do this to use a VSCode plugin called “JSON to TS” to convert the JSON objects to TypeScript interfaces. If you have not installed this plugin install it now:



35. Once installed, open the JSON file on VSCode and “select all” (Ctrl a), then right click to select the “Command Palette”:



36. Type “JSON to TS” in the search box, and select “JSON to TS: Convert from text selection” option:

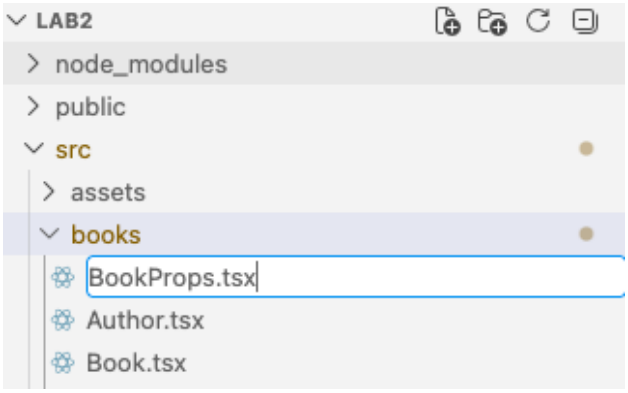


37. You will see the TypeScript interfaces of the data will show on the right-side panel:



38. Now copy the interface data and paste it in a separate text file for later use.

39. Next, inside the 'src/books' folder, create a new file called "`BookProps.tsx`":



40. Paste the interface data you just copied earlier to this file. Export the 'VolumeInfo' interface (line 44), below is my code:



41. Examine the "VolumeInfo", we are only interested in three JSON objects – title (string), subtitle (string), authors (an array of string) and imageLinks (an object with 2 properties both are of type string). Notice that subtitle and imageLinks are optional, which means their type is either string or 'null'.

```
67  interface VolumeInfo {
68    title: string;
69    subtitle?: string;
70    authors: string[];
71    publisher?: string;
72    publishedDate: string;
73    description?: string;
74    industryIdentifiers: IndustryIdentifier[];
75    readingModes: ReadingModes;
76    pageCount?: number;
77    printType: string;
78    categories?: string[];
79    averageRating?: number;
80    ratingsCount?: number;
81    maturityRating: string;
82    allowAnonLogging: boolean;
83    contentVersion: string;
84    panelizationSummary?: PanelizationSummary;
85    imageLinks?: ImageLinks;
86    language: string;
87    previewLink: string;
88    infoLink: string;
89    canonicalVolumeLink: string;
90  }
91
92  interface ImageLinks {
93    smallThumbnail: string;
94    thumbnail: string;
95  }
96
```

42. Next, we will configure the type for our Title component first. It takes a prop named title, which is of string, as follow:

adv-web-dev-app > src > books > Title.tsx > ...

```
1  const Title = ({ title }: { title: string }) => {
2    return <h3>{title}</h3>;
3  };
4
5  export default Title;
6
```

43. The “Image’ component will render an image. It takes 2 props (src and name), both are of type string. However, as we have just seen from the JSON interface above, the ‘imageLinks’ is an optional attribute, which means there could be no such data (i.e. no image source) in the JSON file. We want our books to have an image each for UI consistency. The easiest way to achieve this is to provide a default image. I will use the same image as shown in task 22 as my default image, you can any image if you like. Create a default source to this image inside the Image component (`Image.tsx`)

```
const defaultSrc = "https://books.google.com/books/content?id=S85NCwAAQBAJ&printsec=
```

44. Then we can pass it in the props:

```
3
4   const Image = ({src=defaultSrc, alt}:{src?:string, alt:string}) => {
5     return (
6       <img src={src} alt={alt} />
7     )
8   }
9
10  export default Image
```

Here, we are saying the image source is optional. If there is a value then use that, if not use the default one. We will use the authors’ name as ‘alt’’s value, as we know there is a value for authors, so this prop should be fine.

45. Now we are ready to configure the ‘Book’ component. First import the ‘VolumeInfo’ interface (line 4):

adv-web-dev-app > src > books > Book.tsx > Book

```
1   import Image from "./Image";
2   import Title from "./Title";
3   import Author from "./Author";
4   import type { VolumeInfo } from "./BookProps";
5
```

46. Configure the Book component as follow:

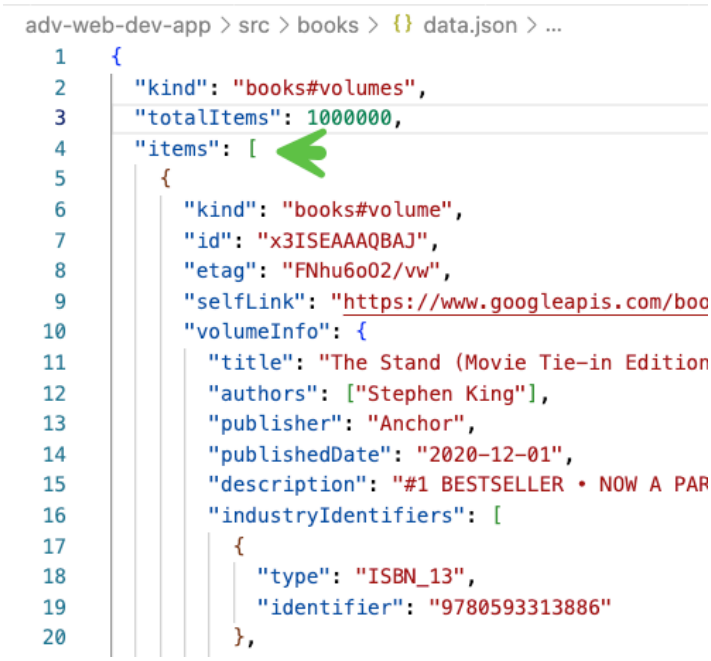
```
8   const Book:React.FC<VolumeInfo> = (props:VolumeInfo) => {
9       const {title, subtitle, authors, imageLinks} = props;
10      console.log(title);
11    return (
12      <article className="book">
13          <Image src={imageLinks?.thumbnail} alt={authors[0]}/>
14          <Title title={title} subtitle={subtitle}/>
15          {authors.map( author => {
16              return <Author name={author}/>
17          })}
18
19      </article>
20    )
21  }
22
23  export default Book
```

Notice that this React.FC stands for REACT Functional Component. It is a built-in TypeScript type used to define the structure of a functional component in a React application. When you use it, it provides automatic type-checking and autocompletion for your component's props while ensuring that the function correctly returns a valid React element. Because we have already taken care of the Image source issue in Image component, so here, we can just pass on the props straight away.

47. Once this is done, we can load the JSON from 'BookList' component. First, import the data (line 2):

```
adv-web-dev-app > src > books > BookList.tsx > ...
1   import Book from "./Book";
2   import data from "./data.json";
3
```

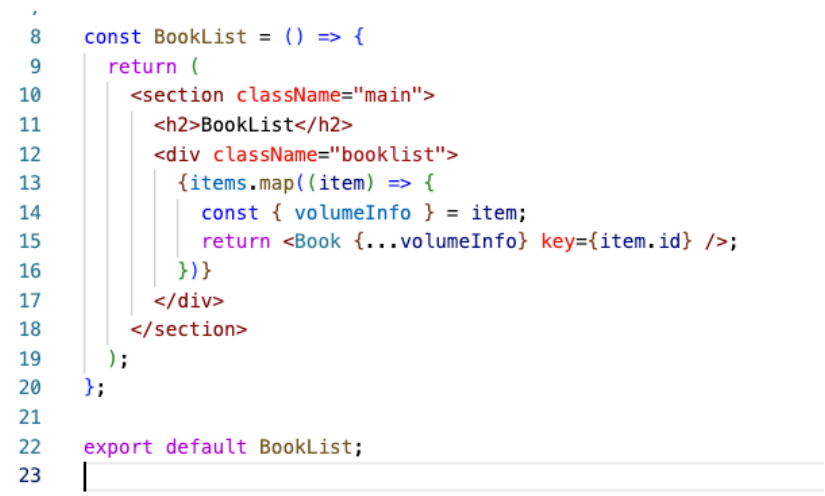
48. Examine the JSON data. The objects that we are interested is stored inside the array called “items”:



49. So, we can unpack the JSON and grab this array as follow (line 4):

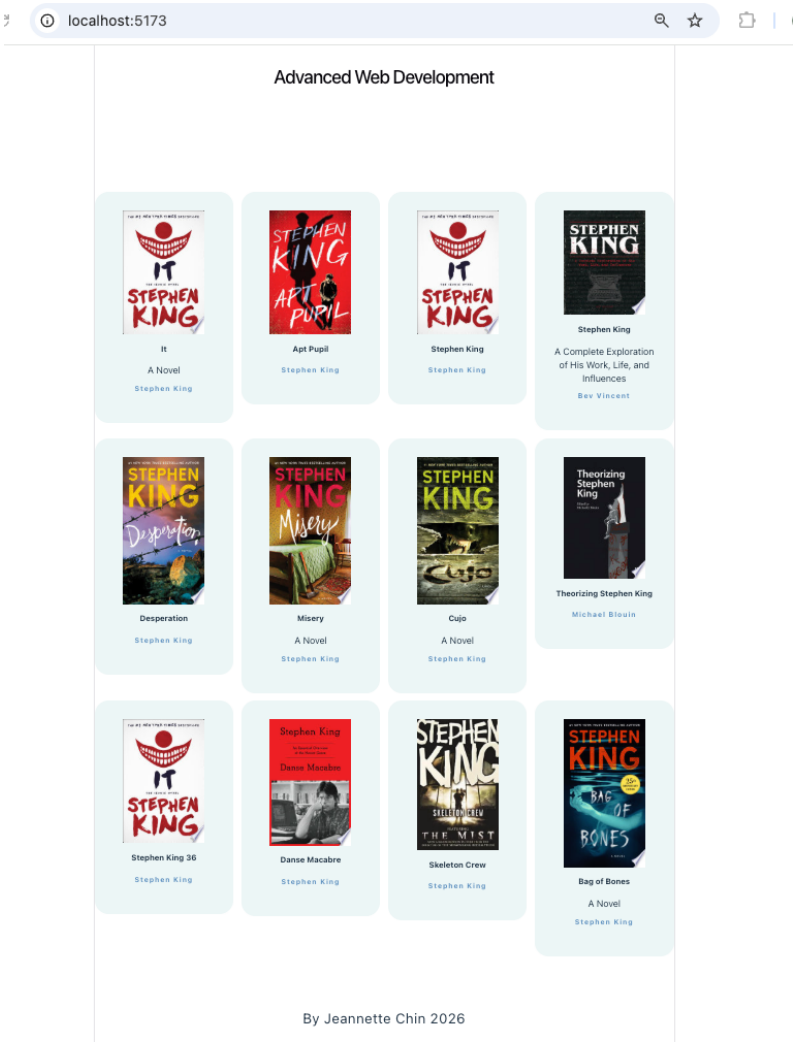


50. Next, we will iterate this array (items) and grab the ‘volumeInfo’ object from each of the item, and pass it unpacked to the child component ‘Book’ as follow:



Note that each children component has a key prop. For this lab we simply assign the item id to this prop (line15).

51. Save the file. Below is my output for your reference:



52. Next task is to push the code to CMP Stugit and name your repo as “<your-username>lab2.0”